

Sports Commentator from Video

HPPCS[05] Capstone Project Report

Abstract

This project develops an automated sports commentary pipeline that analyzes a video without commentary and produces timeline-based commentary in JSON format. The solution samples representative video frames with OpenCV and sends the visual context to two language models. The first model performs event understanding and returns structured information such as the detected action, players when reliably visible, score information when visible, context, and confidence. The second model uses that structured event representation to generate concise, broadcast-style commentary. The two-model separation improves modularity by separating visual/event interpretation from natural-language generation. The output preserves start and end timestamps for every analyzed segment, making the result suitable for post-game review or synchronization with a video player. A deterministic fallback is included when an LLM endpoint is unavailable, allowing the program to complete and produce a valid JSON structure.

1. Introduction

The project objective is to automatically generate live or post-game commentary from sports video using AI. The supplied project guide identifies video understanding, language generation, multimodal integration, and optional speech synthesis as relevant components. This implementation focuses on the required text commentary output and timeline information.

2. Problem Statement

A sports video without commentary contains visual events but does not provide a human-readable narrative. The system must convert temporally ordered visual activity into concise, relevant commentary while avoiding unsupported claims about players, scores, or outcomes.

3. Objectives

- Extract representative frames and time windows from a test sports video.
- Use two LLMs with clearly separated responsibilities.
- Detect and structure observable sports events.
- Generate natural commentary with corresponding start and end times.
- Save the final output in JSON format required by HPPCS[05].

4. Methodology

OpenCV is used for video metadata, frame sampling, and basic motion estimation. LLM #1 receives representative frames and a time-window prompt and returns structured event information. LLM #2 receives the structured event and produces concise commentary. The endpoints are configurable through command-line parameters, so local or hosted OpenAI-compatible model servers can be used. The example configuration uses two different local models through Ollama's OpenAI-compatible interface: Llama 3.2 3B for event understanding and Gemma 3 4B for commentary generation.

5. System Design / Implementation

Module	Function
Video Reader	Reads metadata and creates sequential time windows.
Frame Sampler	Samples representative frames within each window.
LLM #1	Converts visual evidence into structured event information.
LLM #2	Generates broadcast-style commentary from the event.
JSON Writer	Stores timeline, event, confidence, and commentary fields.

6. Results and Analysis

The program produces a JSON timeline in which every segment contains start_time, end_time, detected event, player information when available, score information when available, confidence values, and generated commentary. The project instructions identify BLEU and ROUGE as possible text-quality metrics and user surveys as a subjective assessment method. These metrics can be calculated when reference commentary transcripts are available. In the absence of a provided benchmark transcript in the submission materials, the implementation does not claim a numerical BLEU or ROUGE score.

7. Conclusion and Future Work

The solution demonstrates a complete two-LLM pipeline for generating timeline-based sports commentary from video. Future improvements include sport-specific event models, player and scoreboard recognition, richer temporal reasoning, automatic reference-based BLEU/ROUGE evaluation, streaming inference, and optional text-to-speech output.

8. References

1. HAAI++ Capstone Project Instructions, HPPCS[05] Sports Commentator from Video. 2. Sports Commentator from Video project guide. 3. OpenCV documentation and OpenAI-compatible chat completion interface documentation for the selected local model server.

Acknowledgement

I acknowledge the HAAI++ capstone project guidelines and supplied project materials used to define the scope, input/output requirements, and submission structure.